

WEEK 2 :

Started on: Monday, 2 March 2020, 3:04 PM

State: Pending

Completed on: Monday, 2 March 2020, 3:13 PM

Time taken: 7 mins 5 secs

Name: SATYAVEL C J 2020-0406

1

Correct

Submitted on: 14 Mar

0% Prog

Submission

The general format of if statement is

```
if (condition) {
    statement s1;
    statement s2;
    .....
    statement sn;
}
```

The if construct is a **selection statement**, the statements within the block are executed only once when the **condition evaluates to true**, otherwise the control goes to the first statement after the if construct.

If only one statement is presented in the if construct then there is no need to specify the braces {} i.e. if braces are not specified for the if construct, by default the next immediate statement is the only statement considered for the if construct.

Below code prints the number only when it is **divisible by 3**

```
4 int main()
5 {
6     int num;
7     printf("Enter a number : ");
8     scanf("%d", &num);
9     if (num % 3 == 0)
10     {
11         printf("Given number %d is divisible by 3", num);
12     }
13     return 0;
14 }
```

In the above code, num % 3 == 0 is the **condition**, which verifies whether the **number is divisible by 3**. Only if the condition returns 1 (true) then the control enters in to the **if block** and executes the statement.

Fill in the missing code in the below program to check whether the given number is divisible by 3 or not.

For example:

Input	Result
3	Given number 3 is divisible by 3
7	Given number 7 is not divisible by 3

Answer: (specify regime: 0 %)

```
1 //specify regime: 0%
2
3 int main()
4 {
5     int num;
6     printf("Enter a number : ");
7     scanf("%d", &num);
8     if (num % 3 == 0)
9     {
10         printf("Given number %d is divisible by 3", num);
11     }
12     else
13     {
14         printf("Given number %d is not divisible by 3", num);
15     }
16     return 0;
17 }
```

Input	Expected	Got	
3	Given number 3 is divisible by 3	Given number 3 is divisible by 3	✓
7	Given number 7 is not divisible by 3	Given number 7 is not divisible by 3	✓

Present all tests: ✓

➔ Thus, program is executed successfully

Started on: Monday, 2 March 2020, 3:23 PM

State: Pending

Completed on: Monday, 2 March 2020, 3:37 PM

Time taken: 14 mins 5 secs

Name: SATYAVEL C J 2020-0406

1

Correct

Submitted on: 14 Mar

0% Prog

Submission

Write code which uses an if else statement to check whether a given account balance is greater or lower than the minimum balance.

Use the if else statement and print "balance is low" if the balance is less than **1000**, otherwise print "Sufficient balance".

For example, if the user gives the **input** as 1000:

```
1000
```

Then the program should **print** the result as:

```
Sufficient balance
```

Similarly, if the input is given as 700 then print:

```
Balance is low
```

Hint: Make sure to read the input as a float value.

For example:

Input	Result
1000	Sufficient balance
999.99	Balance is low

Answer: (specify regime: 0 %)

```
1 //specify regime: 0%
2
3 int main()
4 {
5     float balance;
6     scanf("%f", &balance);
7
8     if (balance > 1000)
9     {
10         printf("balance is low");
11     }
12     else
13     {
14         printf("Sufficient balance");
15     }
16     return 0;
17 }
```

Input	Expected	Got	
1000	Sufficient balance	Sufficient balance	✓
999.99	Balance is low	Balance is low	✓

Present all tests: ✓

➔ Thus, program is executed successfully

4

Correct

Accepted out of 100

7 / 100

100%

Write a program which uses an if else statement to verify if the given character is an alphabet or not.

For example, if the user gives the **input** as 'x'.

Then the program should **print** the result as:

Given character 'x' is an alphabet

If the input is given as '7', then print the result as:

Given character '7' is not an alphabet

Note: The ASCII values of alphabets 'A' to 'Z' are 65 to 90 and 'a' to 'z' are 97 to 122.

For example:

Input	Result
x	Given character 'x' is an alphabet
7	Given character '7' is not an alphabet

Answer: (penalty regime: 0 %)

```

1 // Python3 code to verify if the given character is an alphabet or not
2 def main():
3     char = input()
4     if char.isalpha():
5         print("Given character {} is an alphabet".format(char))
6     else:
7         print("Given character {} is not an alphabet".format(char))
8
9 if __name__ == '__main__':
10    main()
11

```

Input	Expected	Got	
x	Given character 'x' is an alphabet	Given character 'x' is an alphabet	✓
7	Given character '7' is not an alphabet	Given character '7' is not an alphabet	✓

Passed all tests: ✓

➔ Thus, program is executed successfully

5

Correct

Accepted out of 100

7 / 100

100%

Write a program which uses an if else statement to verify and print if the given number is an odd or an even.

For example, if the user gives the **input** as 10.

10

Then the program should **print** the result as:

The given number 10 is an even number

If the input is given as 15, then the program should print the result as:

The given number 15 is an odd number

For example:

Input	Result
10	The given number 10 is an even number
15	The given number 15 is an odd number

Answer: (penalty regime: 0 %)

Reset answer

```

1 // Python3 code to verify if the given number is an odd or an even
2 def main():
3     num = int(input())
4     if num % 2 == 0:
5         print("The given number {} is an even number".format(num))
6     else:
7         print("The given number {} is an odd number".format(num))
8
9 if __name__ == '__main__':
10    main()
11

```

Input	Expected	Got	
10	The given number 10 is an even number	The given number 10 is an even number	✓
15	The given number 15 is an odd number	The given number 15 is an odd number	✓

Passed all tests: ✓

➔ Thus, program is executed successfully

Started on: Wednesday, 4 March, 2020, 8:40 AM
 State: Finished
 Completed on: Wednesday, 4 March, 2020, 8:47 AM
 Time taken: 7 mins 27 secs
 Name: 04C7P046, C, 3, 2020, 0406

When an if else construct appear as a statement within another if block or a else block, it is referred to as nesting of if else construct.

Below is an example of a **nested if else** construct:

```

if (expression_1)
{
    if (expression_2)
    {
        statement_1;
    }
    else
    {
        statement_2;
    }
}
else
{
    statement_3;
}

```

In the above syntax, the **statement_2** will be executed only when the conditions in expression_1, expression_2 and expression_3 evaluates to 1 (true).
 [0] is the missing code in the below program to find the **largest** of three numbers using nested if else.

For example:

Input	Result
20 30 77	77 is greater than 20 and 30

Answer: (usually requires 0 %)

```

1 //include <stdio.h>
2
3 int main()
4 {
5     int a, b, c;
6     scanf("%d %d %d", &a, &b, &c);
7     //insert the below code
8     if (a > b)
9     {
10         if (a > c)
11         {
12             printf("%d is greater than %d and %d", a, b, c);
13         }
14         else
15         {
16             printf("%d is greater than %d and %d", c, a, b);
17         }
18     }
19     else
20     {
21         if (b > c)
22         {

```

Input	Expected	Got
20 30 77	77 is greater than 20 and 30	77 is greater than 20 and 30

Passed all tests

➔ Thus, program is executed successfully

The following code uses if else statement to check whether the given integer number is a valid **leap year** or not.

Use if else statement and print "... is a leap year"

- If a year is divisible by 4 and should not be divisible by 100
- If a year is divisible by 400

Otherwise print "... is not a leap year"

[0] is the missing code in the below program to check whether the given year is a **leap year** or not.

For example:

Input	Result
2000	2000 is not a leap year

Answer: (usually requires 0 %)

```

1 //include <stdio.h>
2
3 int main()
4 {
5     int year;
6     scanf("%d", &year);
7     //fill in the missing code
8     if (year % 400 == 0)
9     {
10         printf("%d is a leap year", year);
11     }
12     else if (year % 100 == 0)
13     {
14         printf("%d is not a leap year", year);
15     }
16     else if (year % 4 == 0)
17     {
18         printf("%d is a leap year", year);
19     }
20     else
21     {
22         printf("%d is not a leap year", year);
23     }
24 }

```

Input	Expected	Got
2000	2000 is not a leap year	2000 is not a leap year
2000	2000 is a leap year	2000 is a leap year

Passed all tests

➔ Thus, program is executed successfully

Question 2
Correct
Marked out of 1.00
Flag question

A switch statement is used to change the control flow of a program execution through multiple paths depending on an expression's value. The below code demonstrates how to use a switch-case construct to print the corresponding digit words for the digits 0 to 9, read from the standard input. One way to write using nested if-else for the 10 numbers in the other way is to use a switch-case statement.

Input and output for the below code which demonstrates the usage of switch statement to print the digit word of the given number between 0 to 9.

```

#include <stdio.h>
int main()
{
    int n;
    scanf("%d", &n);
    switch(n)
    {
        case 0:
            printf("Zero\n");
            break;
        case 1:
            printf("One\n");
            break;
        case 2:
            printf("Two\n");
            break;
        case 3:
            printf("Three\n");
            break;
        case 4:
            printf("Four\n");
            break;
        case 5:
            printf("Five\n");
            break;
        case 6:
            printf("Six\n");
            break;
        case 7:
            printf("Seven\n");
            break;
        case 8:
            printf("Eight\n");
            break;
        case 9:
            printf("Nine\n");
            break;
    }
    printf("Number %d is not in the range 0 to 9\n", n);
    return 0;
}

```

Input: 3
Output: Three

Input: 10
Output: Number 10 is not in the range 0 to 9

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int n,i;
6     i = 0;
7
8     if(scanf("%d", &n) != 1)
9     {
10         return 1;
11     }
12
13     printf("The natural numbers from 1 - %d : ", n);
14
15     while(i <= n)
16     {
17         printf("%d ", i);
18         i++;
19     }
20 }

```

Input Expected Got

3	The natural numbers from 1 - 3 : 1 2 3	The natural numbers from 1 - 3 : 1 2 3	✓
9	The natural numbers from 1 - 9 : 1 2 3 4 5 6 7 8 9	The natural numbers from 1 - 9 : 1 2 3 4 5 6 7 8 9	✓

Passed all tests! ✓

➔ Thus, program is executed successfully

Question 1
Correct
Marked out of 1.00
Flag question

Write a C program to print first n natural numbers.

For example, if the user gives the input as :

3

then the program should print the result as:

The natural numbers from 1 - 3 : 1 2 3

For example:

Input	Result
3	The natural numbers from 1 - 3 : 1 2 3
9	The natural numbers from 1 - 9 : 1 2 3 4 5 6 7 8 9

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int n,i;
6     i = 1;
7
8     if(scanf("%d", &n) != 1)
9     {
10         return 1;
11     }
12
13     printf("The natural numbers from 1 - %d : ", n);
14
15     while(i <= n)
16     {
17         printf("%d ", i);
18         i++;
19     }
20 }

```

Input Expected Got

3	The natural numbers from 1 - 3 : 1 2 3	The natural numbers from 1 - 3 : 1 2 3	✓
9	The natural numbers from 1 - 9 : 1 2 3 4 5 6 7 8 9	The natural numbers from 1 - 9 : 1 2 3 4 5 6 7 8 9	✓

Passed all tests! ✓

➔ Thus, program is executed successfully

Question 2
Correct
Marked out of
1.00
Flag question

The below sample code should print Ganga by number of times, where as the input is read by the programmer using `scanf()`.

Fill in the missing code so that it produces the desired output.

For example:

Input	Result
3	Ganga Ganga Ganga

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2 int main()
3 {
4     int i = 0, n;
5     scanf("%d", &n); //Fill the missing code in scanf() function
6     while (i < n)
7     { // complete the condition here
8         printf("Ganga\n"); // write the text to be printed here
9         i = i + 1; // Complete the statement
10    }
11    return 0;
12 }
13
```

	Input	Expected	Got
✓	3	Ganga Ganga Ganga	Ganga Ganga Ganga

Passed all tests! ✓

➔ Thus, program is executed successfully

Question 3
Correct
Marked out of
1.00
Flag question

The below sample code should find the sum of **even numbers** between any two numbers.

[Hint: The numbers should be read by using `scanf()`].

Fill in the missing code so that it produces the desired output.

For example:

Input	Result
3 6	The sum of even integers between the given limits = 10

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int num1, num2, sum = 0;
6     scanf("%d %d", &num1, &num2); // Fill the missing code in the scanf()
7     if (num1 % 2 != 0)
8     { // If it is an odd number then add 1
9         num1 = num1 + 1;
10    }
11    while (num1 <= num2)
12    { // write the condition part
13        sum = sum + num1;
14        num1 = num1 + 2;
15    }
16    printf("The sum of even integers between the given limits = %d", sum);
17    return 0;
18 }
```

	Input	Expected	Got
✓	3 6	The sum of even integers between the given limits = 10	The sum of even integers between the given limits = 10

Passed all tests! ✓

➔ Thus, program is executed successfully

Question 4
Correct
Marked out of 1.00
Flag question

Below partial code is to verify if the given number is a prime number or not.

A prime number is a positive integer greater than 1, which is not divisible by any other number other than 1 and itself. Examples of a few prime numbers are 2, 3, 5, 7, 11, 13, 17, 19, etc.
Fill in the missing code so that it produces the desired output.

For example:

Input	Result
7	The given number 7 is a prime number
119	The given number 119 is not a prime number

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, i = 1, count = 0; // initialize i and count with appropriate values
6     scanf("%d", &n);
7     while (i <= n)
8     { // complete the condition to iterate the loop
9         if (n % i == 0)
10        { // complete the condition to check the remainder is 0 or not
11            count++;
12        }
13        i++;
14    }
15    if (count == 2)
16    { // complete the condition to check the count
17        printf("The given number %d is a prime number\n", n);
18    }
19    else
20    {
21        printf("The given number %d is not a prime number\n", n);
22    }
23 }
```

	Input	Expected	Got
✓	7	The given number 7 is a prime number	The given number 7 is a prime number ✓
✓	119	The given number 119 is not a prime number	The given number 119 is not a prime number ✓

Passed all tests! ✓

➔ Thus, program is executed successfully

Question 5
Correct
Marked out of 1.00
Flag question

Below partial code is to verify if the given number is an armstrong number or not.

An armstrong number is a number that is the sum of its own digits raised to the power of number of digits that make up the original number.

For example, if the given number is 153, the total number of digits are 3, and the sum of cubes of each digit ($1^3 + 5^3 + 3^3$) is equal to the same number 153. Such a number is known as an armstrong number.

Let us take another example, if the given number is 9474, the total number of digits are 4, and the sum of the power of 4 of each digit ($9^4 + 4^4 + 7^4 + 4^4$) is equal to the same number 9474. Such a number is known as an armstrong number.

Similarly,
 $9 = 9^1 = 9$
 $371 = 3^3 + 7^3 + 1^3 = 27 + 343 + 1 = 371$
 $3\ 8208 = 8^4 + 2^4 + 0^4 + 8^4 = 4096 + 16 + 0 + 4096 = 8208$

Fill in the missing code so that it produces the desired output.

For example:

Input	Result
777	The given number 777 is not an armstrong number
9	The given number 9 is an armstrong number

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int number, temp, remainder, i, power, digits = 0, sum = 0;
6     scanf("%d", &number);
7     temp = number;
8     while (temp != 0)
9     { // complete the condition to iterate the loop
10        digits++; // increment the digits
11        temp = temp / 10; // calculate the temp value
12    }
13    temp = number;
14    while (number != 0)
15    { // complete the condition to iterate the loop
16        remainder = number % 10;
17        i = 1;
18        power = 1;
19        while (i <= digits)
20        { // find the powers of each digit
21            power = power * remainder; // calculate power value
22            i++; // increment i value
23        }
24        sum = sum + power;
25        number = number / 10;
26    }
27    if (sum == temp)
28    {
29        printf("The given number %d is an armstrong number\n", number);
30    }
31    else
32    {
33        printf("The given number %d is not an armstrong number\n", number);
34    }
35 }
```

	Input	Expected	Got
✓	777	The given number 777 is not an armstrong number	The given number 777 is not an armstrong number ✓
✓	9	The given number 9 is an armstrong number	The given number 9 is an armstrong number ✓

Passed all tests! ✓

➔ Thus, program is executed successfully

Question 6

Correct

Marked out of 1.00

Flag question

Most of the programming languages provide a special construct/statement using which we can repeatedly execute one or more statement as long as a condition is **true**. In C, we have while, do while and for as the three main looping constructs or statements.

Below is a general syntax for using a while statement:

```
while (condition)
{
    statement_1;
    statement_2;
    ....
}
```

The block of code inside the opening and closing brace which follows the while statement is called the **while-loop body**.

A while statement is used to execute some code repeatedly as long as a condition evaluates to true.

The condition is an expression which should always evaluate to either true or false.

- If it evaluates to true, the body (containing one or more code statements) is executed.
- If the expression evaluates to false, the control skips executing the **while-loop body**.

The while-loop construct is also referred to as an entry controlled loop. Meaning, first the condition is evaluated and only if the condition evaluates to true the body of the loop is executed. After executing the body the control is automatically transferred back to the condition and the process continues until the condition evaluates to false.

See and retype the below code which uses a while-loop to read multiple numbers from standard input and prints their sum when the **sum** exceeds 100.

```
#include <stdio.h>

int main()
{
    int total = 0;
    while (total <= 100)
    {
        int num;
        scanf("%d", &num);
        total += num;
    }
    printf("The total of given numbers is: %d", total);
    return 0;
}
```

For example:

Input	Result
10	The total of given numbers is: 100
10	
10	

Answer: (penalty regime: 0 %)

```
#include <stdio.h>

int main()
{
    int total = 0;
    while (total <= 100)
    {
        int num;
        scanf("%d", &num);
        total += num;
    }
    printf("The total of given numbers is: %d", total);
    return 0;
}
```

Input	Expected	Got
10	The total of given numbers is: 100	The total of given numbers is: 100
10		
10		

Passed all tests! ✓

➔ Thus, program is executed successfully

Question 7

Correct

Marked out of 1.00

Flag question

Fill in the missing code in the below program to read an **integer number** and find the reverse of the given number.

For example if the input is 1234, then the output will be 4321.

Hints

The logic of reversing of any number is pretty simple if you know how to find last digit of any number. Initially the variable reverse contains zero(0), the process of reversing involves four basic steps:

- Multiply the reverse variable by 10.
- Find the last digit of the given number by applying % 10.
- Add the last digit just found to reverse.
- Divide the original number by 10 to eliminate the last digit, which is not needed anymore.

Repeat the above four steps till the original number becomes 0 and finally we will be left with the reversed number in reverse variable.

For example:

Input	Result
1234	The reverse number of a given number = 4321
765	The reverse number of a given number = 567

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, digit, reverse = 0;
6     scanf("%d", &n);
7     while (n > 0)
8     { // write the condition
9         digit = n % 10; // Fill the correct code
10        reverse = (reverse * 10) + digit; // Fill the correct code
11        n = n / 10; // Fill the correct code
12    }
13    printf("The reverse number of a given number = %d", reverse);
14    return 0;
15 }
```

Input	Expected	Got
1234	The reverse number of a given number = 4321	The reverse number of a given number = 4321
765	The reverse number of a given number = 567	The reverse number of a given number = 567

Passed all tests! ✓

➔ Thus, program is executed successfully

Question 8
Correct
Marked out of
1.00
Flag question

Fill in the missing code in the below sample program which finds the factorial of a given number.

Factorial of a non-negative integer n , denoted by $n!$, is the product of all positive integers less than or equal to n .
For example, $5! = 5 * 4 * 3 * 2 * 1 = 120$.

The below sample code computes the factorial of a given non-zero integer.

The main() function declares an integer variable factorial and initializes it to 1, which it will use to store the computed factorial value.

It uses a **while-loop** to iterate from 2 to n multiplying the loop counter in each iteration with the factorial and storing the product again in factorial.

For example:

Input	Result
2	Factorial of given number 2 = 2
4	Factorial of given number 4 = 24

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int i, n, factorial = 1;
6     scanf("%d", &n);
7     i = 2;
8     while (i <= n)
9     { // write the condition
10        factorial = factorial * i; // Fill the correct code
11        i++;
12    }
13    printf("Factorial of given number %d = %d\n", n, factorial);
14    return 0;
15 }
```

	Input	Expected	Got	
✓	2	Factorial of given number 2 = 2	Factorial of given number 2 = 2	✓
✓	4	Factorial of given number 4 = 24	Factorial of given number 4 = 24	✓

Passed all tests! ✓

➔ Thus, program is executed successfully

Question 1
Correct
Marked out of
1.00
Flag question

Fill in the missing code in the below program to print the Fibonacci series i.e., 0 1 1 2 3 5 8 13 21....., up to the limit.

The code in the main() function reads one integer variable n . It uses a for loop to iterate from 0 to n and print the series.

By definition, the first two numbers in the Fibonacci sequence are 0 and 1, and each subsequent number is the sum of the previous two.

For example:

Input	Result
25	The Fibonacci series is : 0 1 1 2 3 5 8 13 21

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2 int main()
3 {
4     int fib1 = 0, fib2 = 1, fib3, n;
5     scanf("%d", &n);
6     printf("The Fibonacci series is : %d\n", fib1, fib2);
7     for (fib3 = fib1 + fib2; fib3 <= n; fib1 = fib2, fib2 = fib3)
8     { // write the initialization, condition and increment part
9         printf("%d ", fib3);
10        fib1 = fib2; // Assign a value
11        fib2 = fib3; // Assign a value
12    }
13    return 0;
14 }
```

	Input	Expected	Got	
✓	25	The Fibonacci series is : 0 1 1 2 3 5 8 13 21	The Fibonacci series is : 0 1 1 2 3 5 8 13 21	✓

Passed all tests! ✓

➔ Thus, program is executed successfully

Question 2
Correct
Marked out of 1.00
Flag question

Fill in the missing code in the below program to check whether the given number is a strong number or not.

A number is called strong number if sum of the **factorials** of its digit is equal to number itself. For example: **145** is considered a strong number since $1! + 4! + 5! = 1 + 24 + 120 = 145$.

The code in the below main() function reads a number from standard input and performs the verification for a strong number by extracting the individual digits and calculating their factorials.

For example:

Input	Result
145	The given number 145 is a strong number
123	The given number 123 is not a strong number

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2 int main()
3 {
4     int rem, n, i, sum = 0, temp, fact = 1;
5     scanf("%d", &n);
6     for (temp = n; n != 0; n = n / 10)
7     { // write the condition part
8         rem = n % 10; // Calculate remainder value
9         fact = 1;
10        for (i = 1; i <= rem; i++)
11        { // write the initialization, condition and increment part
12            fact = fact * i;
13        }
14        sum = sum + fact;
15    }
16    if (sum == temp)
17    { // Fill the condition
18        printf("The given number %d is a strong number\n", temp);
19    }
20    else
21    {
22        printf("The given number %d is not a strong number\n", temp);
23    }
```

Input	Expected	Got	
✓ 145	The given number 145 is a strong number	The given number 145 is a strong number	✓
✓ 123	The given number 123 is not a strong number	The given number 123 is not a strong number	✓

Passed all tests! ✓

➔ Thus, program is executed successfully

Question 3
Correct
Marked out of 1.00
Flag question

Fill in the missing code in the below program to calculate the value of a^n , given two positive non-zero integers a and n.

The code in the main() function reads two integers from standard input and stores them in the variables a and n.

It uses a for-loop to multiply a with itself n number of times.

Variable a_power_n is used to store the computed value of a^n .

After the execution of for-loop is completed, the final value of a_power_n is printed to the standard output.

For example:

Input	Result
2 3	8

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2 int main()
3 {
4     int i, a, n, a_power_n = 1;
5     scanf("%d %d", &a, &n);
6     for (i = 1; i <= n; i++)
7     { // write the initialization, condition and increment part
8         a_power_n = a_power_n * a; // Calculate the value
9     }
10    printf("%d\n", a_power_n);
11    return 0;
12 }
```

Input	Expected	Got	
✓ 2 3	8	8	✓

Passed all tests! ✓

➔ Thus, program is executed successfully

Question 4
Correct
Marked out of 1.00
Flag question

Fill in the missing code in the below program to verify whether the given number is perfect, abundant or deficient.

A number is said to be perfect if it equals the sum of its proper divisors. For example, 6 and 28 can be called **perfect numbers** as : $6 = 1 + 2 + 3$ and $28 = 1 + 2 + 4 + 7 + 14$.

Alternatively, if the sum of a number's proper divisors **exceeds** the number itself, it is said to be abundant, while if the sum of a number's proper divisors is **less than** the number itself, it is said to be deficient.

For example:

Input	Result
6	The given number 6 is a perfect number
10	The given number 10 is a deficient number
12	The given number 12 is an abundant number

Answer: (penalty regime: 0 %)

Reset answer

```
1 #include <stdio.h>
2 int main()
3 {
4     int n, i, sum = 0;
5     scanf("%d", &n);
6     for (i = 1; i < n; i++)
7     { //Write the initialization, condition and increment part
8         if (n % i == 0)
9         { // Fill the condition
10             sum = sum + i;
11         }
12     }
13     if (sum == n)
14     { // Fill the condition
15         printf("The given number %d is a perfect number", n);
16     }
17     else if (sum < n)
18     { // Fill the condition
19         printf("The given number %d is a deficient number", n);
20     }
21     else
22     {
```

	Input	Expected	Got	
✓	6	The given number 6 is a perfect number	The given number 6 is a perfect number	✓
✓	10	The given number 10 is a deficient number	The given number 10 is a deficient number	✓
✓	12	The given number 12 is an abundant number	The given number 12 is an abundant number	✓

Passed all tests! ✓

➔ Thus, program is executed successfully

Question 5
Correct
Marked out of 1.00
Flag question

A for loop is used to iterate over a range of values using a loop counter, which is a variable taking a range of values in some orderly sequence (eg., starting at 0 and ending at 10 in increments of 1). The value stored in a loop counter is changed with each iteration of the loop, providing a unique value for each individual iteration. The loop counter is used to decide when to terminate the loop.

The value stored in a loop counter is changed with each iteration of the loop, providing a unique value for each individual iteration. The loop counter is used to decide when to terminate the loop.

A for loop construct can be formed as an entry control loop.

Below is the syntax of a for loop.

for (initialization, condition, update)

{

statement(s);

}

1. The initialization expression initializes the loop counter. It is executed **once** at the start of the loop.

2. The condition expression is evaluated as long as the condition expression evaluates to true.

3. The update expression is executed after each iteration through the loop, to **increment, decrement or change the loop counter**.

Example with code:

```
for (i = 0; i < 10; i++)
{
    printf("%d\n", i);
}
```

1. Above for loop statement initializes an integer variable i (which is the **loop counter**) as part of the initialization expression.

2. In the update section, it increments the variable i by using the post-increment expression i++.

3. The expression in condition is i < 10. The for loop keeps on executing the code inside the loop body as long as this condition evaluates to true. And the loop terminates when the condition evaluates to false.

4. As a good practice to always keep the loop body (which contains the code to be executed) within an opening brace (and a closing brace).

Note: for: at the end of the for statement.

Complete the below code to check your understanding of the for loop syntax. The completed code should print numbers from 10 to 20, one per line.

For example:

Result

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

Question 6
Correct
Marked out of 1.00
Flag question

Write a program to read **n** numbers from the user and then count number of "Odd" and "Even" numbers.

Constraints:

- $1 \leq n \leq 10^6$
- $10^{-3} \leq \text{elements} \leq 10^3$

Sample test case:

3-----> First line of input is n i.e. 3.

5 6 7-----> Second line of input is n space separated integer values/elements.

Even: 1-----> Third line prints the output (the count of even elements).

Odd: 2-----> Fourth line prints the output (the count of odd elements).

Note: Do use the **printf()** function with a **newline** character (**\n**) to print your results on newline.

Instruction: To run your custom test cases strictly map your input and output layout with the visible test cases.

For example:

Input	Result
3	Even: 1
5 6 7	Odd: 2

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n;
6     int num;
7     int even = 0, odd = 0;
8
9     scanf("%d", &n);
10
11     for(int i = 0; i < n; i++)
12     {
13         scanf("%d", &num);
14
15         if (num % 2 == 0)
16             even++;
17
18         else
19             odd++;
20     }
21     printf("Even: %d\n", even);
22     printf("Odd: %d\n", odd);
```

Input	Expected	Got
3	Even: 1	Even: 1
5 6 7	Odd: 2	Odd: 2

Passed all tests! ✓

➔ Thus, program is executed successfully

Question 7
Correct
Marked out of 1.00
Flag question

Write a program to find **sum** and **mean** of **n** numbers.

Constraints:

- $1 \leq n \leq 10^6$
- $10^{-3} \leq \text{elements} \leq 10^3$
- Result of mean should print upto **2 decimal places**.

Sample test case:

4-----> First line of input is the value on n.

3 5 7 8-----> Second line of input is n space separated integer values.

Sum: 23-----> Third line prints the Sum as required.

Mean: 5.75-----> Fourth line prints the Mean as required.

Instruction: To run your custom test cases strictly map your input and output layout with the visible test cases.

For example:

Input	Result
4	Sum: 23
3 5 7 8	Mean: 5.75

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, num;
6     int sum = 0;
7
8     scanf("%d", &n);
9
10    for(int i = 0; i < n; i++)
11    {
12        scanf("%d", &num);
13        sum = sum + num;
14    }
15
16    float mean = (float)sum / n;
17
18    printf("Sum: %d\n", sum);
19    printf("Mean: %.2f\n", mean);
20
21    return 0;
```

Input	Expected	Got
4	Sum: 23	Sum: 23
3 5 7 8	Mean: 5.75	Mean: 5.75

Passed all tests! ✓

➔ Thus, program is executed successfully

